



Git Good: an Introduction to GitHub for Collaboration

This document is your guide to using GitHub.

Each exercise described by our instructors is accompanied by steps, descriptions and illustrations in this packet.

Use this guide as you wish but note that the instructions listed here may differ from the directions given during your workshop.

Key Vocabulary

Icon on GitHub	Term	Description
	Repository (repo)	Filing cabinet / directory (is used to store your project)
	Commit	Save point
	Issue	Feedback and “to-do” list (can also be used to track tasks)
	Branch & Fork	Different “streams” of work
	Pull request	Ask to incorporate your changes
	Merge	Incorporate changes

Create a new GitHub account at <https://github.com/>



Exercise 1 - Commenting on an issue

Background

GitHub issues are an accessible way to contribute to a project. They are often used by project teams to keep track of what they're working on and receive feedback from others.

Issues are like a message thread, which you can connect with other parts of the GitHub workflow (for example, they can be referenced in commit messages). You can tag other project members to get their input, and use emojis to show you care!

Reading and making issues

Issues are used to keep track of your work on GitHub. They can be used to track ideas, feedback, tasks, or bugs in code, and more.

In many open source projects, issues can be labelled “good-first-issue” to identify feedback or tasks that can be actioned by folks who are new to that project.

Issues can be viewed by anyone in the project if the repository is private, or anyone in the world if the repository is public!

Anyone with a GitHub account can comment on a public issue. This visibility is important for transparency (showing your working) and accountability (taking responsibility for what you do).

Task

1. Find the issue in the repository for this workshop (<https://github.com/The-Turing-Summer-Experience-2026/github-introduction>) and follow the instructions in the issue.
2. Note that issues can be written in markdown.
 - a. Take a look at this Markdown Cheat Sheet (<https://www.markdownguide.org/cheat-sheet/>).
 - b. Use the “write” tab in the issue comments to write using markdown syntax, then the “preview” tab to see how it is rendered.



Exercise 2 - Edit a file and make a pull request

Background

- To contribute to a collaborative project, you need to be able to:
 - edit the file you want to make changes to,
 - keep that change in a separate branch, and
 - ask your collaborators to review your work before bringing it into the main branch of the repository.

On GitHub, this is done through creating a **new branch** and then a **pull request**.

Task

Forking a repository

To prevent making unnecessary changes to a repo you can create a **fork**. This creates a new repo with the same code as the original repo. The cleverness is that this repo can be used to iterate on ideas without affecting the original, and any changes can be *proposed* back to the upstream repository to be reviewed and potentially accepted via a **pull request**.

1. Go to the *github-introduction* repo: <https://github.com/The-Turing-Summer-Experience-2026/github-introduction/>
2. In the top right hand corner click **Fork**
3. In the menu, choose an owner for the forked repo (this will be you) and create the fork.

Make a commit on a branch (see below for more details on committing)

1. Go to your fork of the repo
2. Navigate to the *introductions* file in the *github-workshop* folder
3. Click on the pencil icon to edit
4. Find the line you have been assigned by the instructor and add your answer to the question.
5. Scroll up to the 'Commit changes' green button at the top right.
6. Write a good commit message! - type a message that says what you have changed.
7. Create a new branch for the commit.
8. Click on 'Propose changes' - this creates a commit and sets up the pull request.
9. Don't close this page



Committing - or saving - your changes

Committing your changes is like hitting the “save button” for a file. GitHub will not automatically save your changes, so it’s important not to skip this step.

Whatever changes you have made in the file will be deposited into your repository.

It is good practice to write a descriptive commit title and a short description of what you have done in the commit changes box.

So something like - commit title: ‘first edit of the readme file’; description: ‘copied template from ... and edited it with the details of this project’. This information about the commit is called a “commit message”, and the commit title will enable you to quickly look through the history of changes for a file (which is why making them descriptive is so important - it’s like leaving a helpful note to your future self).

You can see a list of your commits (or your “commit history”) by clicking the clock symbol on your landing page or within the page for each file.

Create a Pull request

IMPORTANT: When opening your pull request, make sure you are comparing back to the correct repository – it should be called **The-Turing-Summer-Experience-2026/github-introduction**.

1. Go back to the pull request page.
2. You should see a green check at the top, saying ‘Able to merge’.
3. Add any additional information in the ‘Leave a comment’ section, if you like.
4. Click on ‘Create a pull request’.



5. Assign the co-facilitator for the day as reviewer - *this is the person who said hello to you in the Exercise 1 issue*.

Open a pull request

The change you just made was written to a new branch named `eirini-zormpa-patch-1`. Create a pull request below to propose these changes.

base: `main`

←

compare: `eirini-zormpa-patch-1`

✓ Able to merge. These branches can be automatically merged.



Create README.md

Write

Preview

H B I        

Leave a comment

Attach files by dragging & dropping, selecting or pasting them.

Create pull request



Conversation 0

Commits 1

Checks 0

Files changed 1

+1 -0



eirini-zormpa commented 19 minutes ago

Member

No description provided.

Create README.md

Verified

52dfc8a

Add more commits by pushing to the `eirini-zormpa-patch-1` branch on `aim-rsf/training`.

This branch has not been deployed

Request up to 16 reviewers

ra

BrainonSilicon Sophia Batchelor

Rainiefantasy Mahwish M

RayStick Rachael Stickland

Licenses

It is important that all of your work has a license from the very beginning or no one can reuse it. Licenses tell other researchers how they are able to reuse, modify and remix your work. No license implies that others are *not* allowed to use your work, even with attribution. So it is better to include a license that lets people know what they can and can't do and how to give you credit for your work.

You can find more information about licenses in the [Licensing](#) chapter of The Turing Way.



Types of Open Licenses

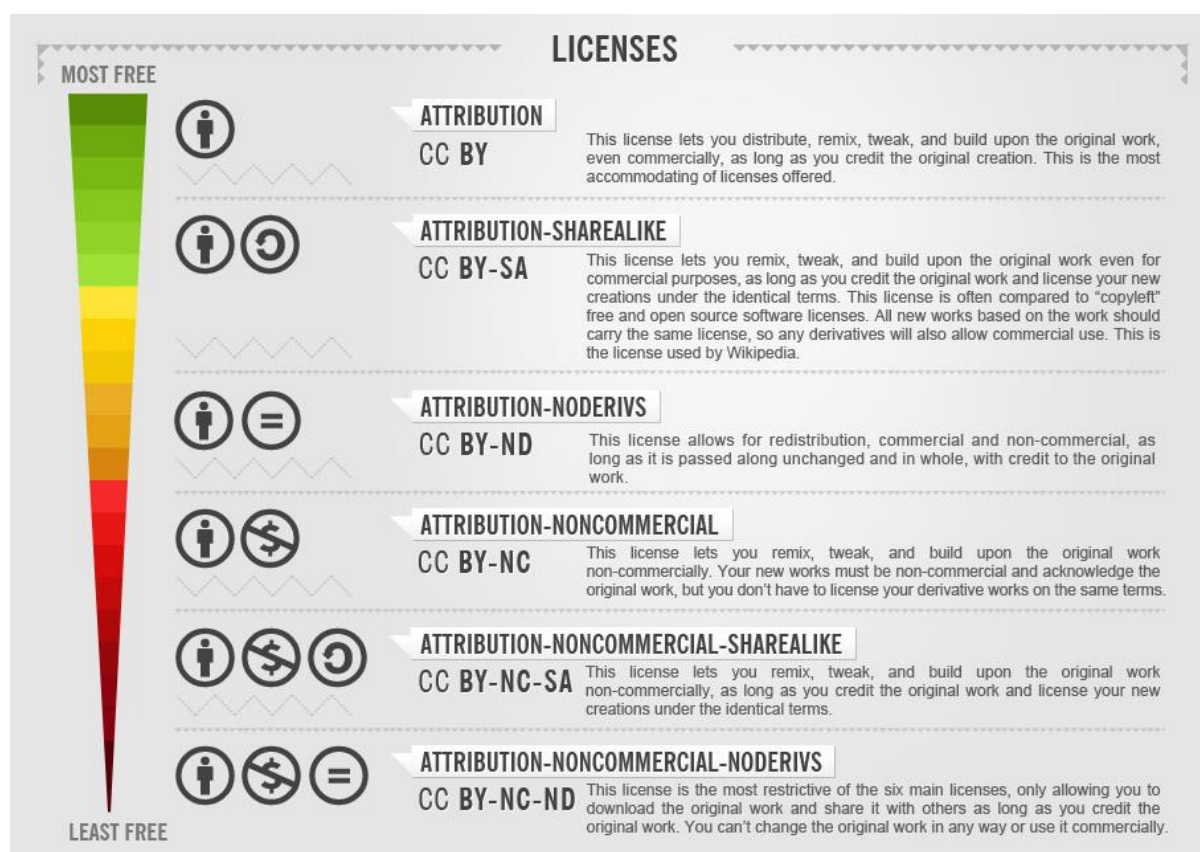
Licenses affect whether and how you can use, modify, and share work. Materials that you share can be in the public domain, use open licenses, or have copyright associated with them. This section has more information about open licenses you can use for your work.

As someone working with open materials, you have many different options for the types of licences you can use for your project.

One popular type of creative licence is the Creative Commons (CC) licence, which offers many options that affect how materials can be used by others. This license is most often used for photos, but also can be used with other types of materials.

For example, depending on your field, much of your work may be documents with only some data or code. The standard licenses offered on GitHub are most appropriate for software and won't really be the right kind for documents.

The following graphic, created by the Creative Commons team can help you to understand the different types of Creative Common Licenses that you can use:



Source: Creative Commons (CC BY-SA)



Software, on the other hand, requires another type of license. Similar to Creative Commons licensed photos or other assets, software requires licenses in order to be used by others.

Each type of software license has its own terms and conditions, which can significantly impact how software can be used, modified, and distributed. While there are many different types, but here are a few examples:

Public Domain Software	Permissive	Copyleft	Proprietary
Anyone is free to use and modify the software Software is not owned by anyone. Users can use, modify, and distribute the software without any restrictions.	Establishes some requirements for distribution or modification of the software. Requires preserving license notices, copyrights, or trademarks.	A general method for making a program (or other work) free, and requiring all modified and extended versions of the program to be free as well.	Software is owned by an individual or company. Users purchase the right to use the software. Source code is not available to users.
No license	MIT License Apache License BSD License	GNU General Public License (GNU GPL)	Proprietary License (No Public License)
Examples: Some older software, government publications	Examples: jQuery, Rails	Examples: Linux, WordPress	Examples: Microsoft Windows, Adobe Photoshop.

Licence

This material has been created using the 2024-TTW-widening-participation-Gitgood-handout-v2.pdf, which can be found on Zenodo at DOI:

<https://doi.org/10.5281/zenodo.12802974>

The original materials (unless otherwise specified) are available under the Creative Commons Attribution 4.0 Licence. Please see the [human-readable summary](#) of the CC BY 4.0 and the full [legal text](#) for further information.

